

Forced authentication redirection

```
if (!auth.check()) {  
  auth.forceAuth(); }  
else {  
  setVote(--);  
  ...  
}
```

Conditionally perform effects

```
switch (pType,result){  
| ("post",None) => ()  
| ("post",r) =>  
  Js.Promise.resolve(r)  
  >= Model.Decode.Response  
  .fetchComments  
  >= ...  
}
```

Continuation-passing style

All of above are examples of *continuation* [Ler, Ler] and how they're implicitly tracked. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut. This can be reified using the concept of *CPS (continuation-passing style)*, where Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri.

Monad for continuation-passing style

```
result |> Option.iter @@ r =>  
Js.Promise.resolve(r)  
>= Model.Decode.Response  
  .fetchComments  
>= ...
```

where Option is

Literature

Bonsai [Bon] is a library for general-purpose reactive programming and with a focus on reactive user interface. It belongs to the same category of reactive-programming libraries like Reason React. One of the central design points

of Bonsai is that it has an applicative programming interface. This applicative enforces the concept of incremental computation—through 't Computation.t—and *generalized side effects*—through 't Effect.t—which can mean an async operation ('t Lwt.t) or Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.. And users have to use specific let-bindings to write with these applicatives. This applicative-based design is very strict. Therefore, a consequence is that developers require a learning curve to use this library.

[Ler] X. Leroy, “Continuations and the CPS transformation,” in *Control structures in programming languages: from goto to algebraic effects*, ch. 6.

[Ler] X. Leroy, “Programming with continuations,” in *Control structures in programming languages: from goto to algebraic effects*, ch. 7.

[Bon] “Bonsai—A library for building dynamic webapps, using Js_of_ocaml.” [Online]. Available: <https://github.com/janestreet/bonsai>